

Testing de Software 2024

Práctico 3

Github assignment: <https://classroom.github.com/a/9ErbtzuM>

Ejercicio 1. Leer el capítulo 6 del libro “*Introduction to Software Testing*” de Ammann y Offutt (2da edición).

Ejercicio 2. Retome el ejemplo visto en la teoría:

```
/**
 * Counts the number of occurrences of a specified element in a given list.
 *
 * This method iterates through the provided list to count how many times
 * the specified element appears. If the list or the element is null,
 * an IllegalArgumentException is thrown.
 *
 * @param l the list in which to count the occurrences of the specified element
 * @param element the element whose occurrences need to be counted
 * @return the number of times the specified element appears in the list
 * @throws IllegalArgumentException if the list l or the element element is null
 */
public static int numberOfOccurrences(List<Integer> l, Integer element); {
```

Tome los Modelos del Dominio de Entradas (MDE) presentados en la teoría. Liste los requisitos de test que se deben cubrir para lograr cobertura de pares (PWC). Provea test (en `assignment3_exercises.numberOfOccurrencesTest.java`) para cubrir los requisitos. En este punto, la documentación de los test será muy importante: Cada test creado debe estar acompañado por un comentario indicando que requisito/s cubre. Utilice `setUp`, `tearDown` y test parametrizados siempre que sea necesario. Implemente la rutina `numberOfOccurrences` de tal manera que todos sus test pasen.

Ejercicio 3. Responda las siguientes preguntas para el método `intersection(...)` y el MDE que se muestra a continuación:

```
/**
 * Computes the intersection of two sets of integers.
 *
 * This method returns a new set containing only the elements that are present
 * in both of the input sets. The input sets themselves are not modified.
 *
 * @param set1 the first set of integers
 * @param set2 the second set of integers
 * @return a new set containing the intersection of set1 and set2
 * (i.e., the elements that are common to both sets)
 * @throws NullPointerException if either set1 or set2 is null
 */
public static Set<Integer> intersection (Set<Integer> set1, Set<Integer> set2);
```

ID	Características	b1	b2	b3	b4
C1	Validez de set1	set1 = null	set1 = {}	set1 tiene al menos un elemento	
C2	Validez de set2	set2 = null	set2 = {}	set2 tiene al menos un elemento	
C3	Relación entre set1 y set2	set1 y set2 representan el mismo conjunto	set1 es un subconjunto de set2	set2 es un subconjunto de set1	set1 y set2 no tienen elementos en común

a) ¿La partición para C1 (Idem. C2) satisface la propiedad de completitud? Si la respuesta es negativa dar un contraejemplo.

b) ¿La partición para C1 (idem. C2) satisface la propiedad de no solapamiento? Si la respuesta es negativa dar un contraejemplo.

c) ¿La partición para C3 satisface la propiedad de completitud? Si la respuesta es negativa dar un contraejemplo.

d) ¿La partición para C3 satisface la propiedad de disjointness? Si la respuesta es negativa dar un contraejemplo.

e) Revise las características para eliminar cualquier problema que encuentre.

f) Provea los requisitos de test para satisfacer cobertura de bloques bases (definir bloque base para cada partición). Provea tests (en `assignment3_exercises.IntersectionTest.java`) para cubrir los requisitos. En este punto, la documentación de los test será muy importante: Cada test creado debe estar acompañado por un comentario indicando que requisito/s cubre. Utilice *setUp*, *tearDown* y test parametrizados siempre que sea necesario. Implemente la rutina `intersection` de tal manera que todos sus test pasen.

Ejercicio 4. Considere el problema de buscar un patrón dado en un *String*. Una posible implementación (con su correspondiente especificación) se encuentra en la clase `assignment3_exercises.PatternIndex`. Usando la técnica de particionado del espacio de entradas y el criterio Pair-Wise Coverage derive tests para la implementación de `PatternIndex`. Implemente los tests. Utilice *setUp*, *tearDown* y test parametrizados siempre que sea necesario. ¿Descubrieron sus tests fallas en la implementación? Recuerde la importancia de documentar los test.

Ejercicio 5. La siguiente tabla contiene un MDE para la interface `java.util.Iterator` (sección 6.4 de la bibliografía (<https://docs.oracle.com/javase/7/docs/api/java/util/Iterator.html>)).

Nombre del método	Parámetros	Tipo de retorno	Posibles valores de retorno	Excepciones	ID Car.	Característica	Partición	Cubierto por
<code>hasNext()</code>	Iterator state	boolean	True, false		C1	Iterator tiene más valores	{true,false}	
<code>next()</code>	Iterator state	E(element)	E, null		C2	Iterator retorna un objeto no null	{true,false}	
				<code>NoSuchElementException</code>				C1
<code>remove()</code>	Iterator state			<code>UnsuportedOperationException</code>	C3	<code>remove()</code> es soportado	{true,false}	
				<code>IllegalStateException</code>	C4	Restricción de <code>remove()</code> se satisface	{true,false}	

	<code>hasNext()</code>	<code>next()</code>	<code>remove()</code>
C1	X	X	X
C2		X	
C3			X
C4			X

La columna "Parámetros" muestra que el comportamiento de los tres métodos está determinado por el "estado del iterador", el cual es complejo. Primero, el estado contiene los valores que el iterador aún debe devolver mediante llamadas posteriores a `next()`. Si no existen tales valores, `hasNext()` devolverá falso.

En segundo lugar, el estado contiene información sobre si se puede llamar exitosamente a `remove()` y, de ser así, qué objeto se debe eliminar y de qué colección subyacente. Por lo tanto, la colección subyacente también es parte del estado del iterador.

Derive test para satisfacer el criterio de cobertura de pares, utilizando el MDE dado (consulte la bibliografía si lo cree necesario). Tener en cuenta que `Iterator` es una interfaz y las interfaces Java no son directamente instanciables. Por lo tanto, para desarrollar test concretos, utilice `ArrayList` como colección subyacente.